Sprint 4 · Reto «Herramientas internas»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 En pocas palabras (léelo primero)

Qué os pide el cliente. El equipo de reservas de TurisData calcula cada semana sus cifras **a mano en una hoja de cálculo**: es lento y cada persona lo hace distinto. Os pide un **programa en Python** que lea el fichero de reservas, calcule **tres cifras clave (KPI)**, avise con claridad si el fichero está mal y guarde el resultado en un fichero JSON que otros sistemas puedan leer.

Qué vais a hacer, en cinco pasos:

1. **Preparar** la carpeta de la plantilla y ver que las pruebas fallan (es lo normal).
2. **Rellenar** las funciones de `kpis.py` una a una hasta que las pruebas pasen.
3. **Ejecutar** el programa con el CSV real y guardar `salida.json`.
4. **Romperlo a propósito** y comprobar que avisa con claridad.
5. **Escribir** el README y subirlo todo a Git.

Qué vais a entregar: una carpeta `sprint-04/` en el repositorio del equipo con `kpis.py`, `test_kpis.py`, `README.md` y `salida.json`.

Con qué herramientas: Python 3.11 (solo biblioteca estándar), un editor o Colab, una terminal, Git y GitHub. Detalle en «Con qué herramientas».

Cuánto tiempo: en clase, 10 min de planning (día 1) y **100 min de reto + 15 min de entrega** (día 2). Lo que no quepa va a casa o a la 6.ª hora.

Núcleo y extra. Núcleo: las funciones obligatorias hasta que `test_kpis.py` pase, y el README. Extra: pruebas propias, versión con pandas y `--por-mes`.

2 Los tres KPI

KPI	Cómo se calcula	Detalle importante
Ocupación aproximada (proxy)	noches de reservas **no canceladas** ÷ (días del periodo × habitaciones) × 100	El periodo va de la primera a la última fecha de llegada. Es un **proxy**: todas las noches se cuentan el día de llegada y la capacidad la suponéis vosotros (`--capacidad`, por defecto 40).
Ingreso medio por reserva	media de `noches × precio_noche` de las reservas **no canceladas**	Las canceladas no generan ingreso. 2 decimales.
Tasa de cancelación por canal	canceladas ÷ reservas del canal × 100	Una tasa por cada canal. 1 decimal.

Estas fórmulas están también en los docstrings de `kpis.py`: es la única definición, no hace falta inventar otra.

3 Qué hay que entregar

Fichero	Qué contiene	Cómo se sabe que está bien
`kpis.py`	El programa: funciones + `main`. Solo biblioteca estándar.	`python test_kpis.py` pasa todas las pruebas
`test_kpis.py`	Las pruebas. Se dan hechas; **no se modifican** (podéis añadir las vuestras aparte).	—

Fichero	Qué contiene	Cómo se sabe que está bien
`README.md`	Cómo se ejecuta, cómo se prueba, definición de cada KPI, por qué la ocupación es un proxy, uso de la IA. Partid de `README_PLANTILLA.md`.	Una persona ajena al equipo puede ejecutarlo solo con él
`salida.json`	El resultado generado con el **CSV real**	Tiene los tres KPI y la sección `calidad`

Dónde se sube: a la carpeta `sprint-04/` del repositorio privado del equipo (cómo, en Cómo entregar en GitHub). Con **al menos 3 commits** con mensaje claro, hechos por personas distintas.

Hasta dónde llegar. El equipo decide el nivel; el mínimo es una entrega válida.

Nivel	Qué hay que tener	Nota*
Mínimo	`leer_reservas`, `tasa_cancelacion_por_canal` e `ingreso_medio_por_reserva` pasan sus pruebas · `salida.json` creado · mensaje claro si falta el fichero · README corto · 3 commits (la ocupación y `main`, con ayuda del docente)	2
Estándar	Todas las pruebas en verde · errores controlados · docstrings y PEP 8 · README con definiciones y límites del proxy · declaración del uso de IA	3
Extra	Pruebas propias · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	4

*Orientativo; puntúa el docente con la rúbrica. La lista de comprobación del final de la ficha describe el nivel **estándar**.

4 Con qué herramientas

Herramienta	Para qué	Cómo conseguirla
Python 3.11 (biblioteca estándar: `csv`, `json`, `argparse`, `datetime`)	Escribir y ejecutar `kpis.py`	Ya instalado en el aula; si no, el de Colab. **Pandas solo en el extra.**
Editor y terminal: VS Code (recomendado) o Colab	Editar el código y ejecutar `python test_kpis.py`	En Colab subid la carpeta y poned `!` delante: `!python test_kpis.py`
La plantilla `reto_herramientas/`	`kpis.py` con funciones vacías, `test_kpis.py`, `README_PLANTILLA.md`	Descargadla (en GitHub: *Code → Download ZIP*) y trabajad en una copia
El CSV real `reservas_turisdata.csv`	6000 reservas **sintéticas**	Descargadlo (botón *Download raw file*) y **dejadlo junto a `kpis.py`**
Git y **GitHub**	Guardar versiones (*commits*) y entregar	Terminal o VS Code. Si Git no funciona, subid los ficheros desde la web de GitHub (*Add file → Upload files*): cada subida es un commit.
Los cuadernos del sprint	Las piezas que os faltan	`S04_03` (diccionarios y listas), `S04_04` (CSV, JSON y excepciones), `S04_05` (Git)
Un asistente de IA (opcional)	Entender errores, pedir pistas	Con las reglas de «Cómo usar la IA»

5 Guía paso a paso

Total en clase: 100 minutos de reto + 15 de entrega. Cada paso dice cuánto dura, qué se hace y cuándo está hecho.

Paso 1 · Preparar la carpeta y ver que las pruebas fallan (día 2, 10 minutos)

1. Copiad la carpeta `reto_herramientas/` y dejad junto a `kpis.py` el CSV real.
2. Abrid una terminal en esa carpeta y ejecutad `python test_kpis.py`.
3. Repartid el trabajo (ver el paso 2).

Hecho cuando: veis las pruebas fallando con `NotImplementedError`. Es lo normal: las funciones están vacías.

Paso 2 · Rellenar las funciones, de una en una (día 2, 45 minutos)

1. **Entre todo el equipo, primero `leer_reservas`** (con `_convertir_fila`): las demás pruebas dependen de ella.
2. Después, repartid el resto: una persona por función, y otra la revisa.

3. Para cada función: programadla, ejecutad `python test_kpis.py`, mirad qué prueba falla, corregid y repetid. Es el método de los profesionales: trabajar **guiados por las pruebas**.
4. Nunca cambiéis `test_kpis.py` para que pase: el código es el que se adapta a las pruebas.

Función	Qué hace	Prueba que la valida	Dónde está la pista
`_convertir_fila` y `leer_reservas`	Lee el CSV, convierte los tipos, **descarta y cuenta** las filas defectuosas	`test_leer_reservas…`, `test_fichero_inexistente`, `test_faltan_columnas`, `test_filas_defectuosas…`, `test_valores_imposibles…`	Cuaderno `S04_04`
`tasa_cancelacion_por_canal`	% de canceladas de cada canal, 1 decimal	`test_tasa_cancelacion…`	Cuaderno `S04_03`
`ingreso_medio_por_reserva`	Media de `noches × precio` de las no canceladas, 2 decimales; `0.0` si no hay	`test_ingreso_medio…`	Cuaderno `S04_03`
`ocupacion_proxy`	La fórmula de la tabla de KPI	`test_ocupacion_proxy`	Docstring de la función

Hecho cuando: esas pruebas pasan. En el nivel mínimo, la ocupación se completa con ayuda del docente.

Paso 3 · Exportar el resultado y ejecutar el programa entero (día 2, 15 minutos)

1. Programad `exportar_json` (UTF-8, `ensure_ascii=False`, sangría de 2) y `main` (leer → calcular → añadir `calidad` → exportar). `calcular_kpis` ya viene hecha.
2. Ejecutad: `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`
3. Abrid `salida.json` y comparadlo con este resultado esperado:

```
{
  "reservas_analizadas": 6000,
  "ocupacion_proxy_pct": 42.0,
  "ingreso_medio_por_reserva": 576.04,
  "tasa_cancelacion_por_canal_pct": {"empresa": 43.3, "directo": 31.5, "web_intermediaria": 53.9, "agencia": 44.0},
  "calidad": {"filas_validas": 6000, "filas_descartadas": 0, "avisos": []}
}
```

Hecho cuando: existe `salida.json` con las mismas cifras y `python test_kpis.py` pasa todo. Si el vuestro difiere, mirad «Errores frecuentes».

Paso 4 · Probar que no se rompe (día 2, 15 minutos)

1. Ejecutad el programa con un fichero que no existe.
2. Quitad una columna al CSV (en una copia) y ejecutad de nuevo.
3. Poned un dato incorrecto en una fila (por ejemplo, un precio con letras).
4. Comprobad en cada caso que el mensaje es claro, sin el texto rojo de `Traceback`:

Caso	Mensaje esperado	Código de salida
1. Fichero que no existe	`ERROR: No se encuentra el fichero: …`	1
2. Falta una columna	`ERROR: Faltan columnas obligatorias: […]`	1
3. Dato incorrecto en una fila	El programa sigue y la fila aparece en `calidad.avisos`	0

Hecho cuando: los tres casos se comportan así. Ese será vuestro «provocar un error» en la review.

Paso 5 · README y Git (día 2, 15 minutos)

1. Copiad `README_PLANTILLA.md` como `README.md` y rellenadlo: cómo se ejecuta y se prueba, cada KPI, por qué la ocupación es un proxy y el uso de IA.
2. Subid el trabajo a `sprint-04/` con **al menos 3 commits**, hechos por personas distintas.
3. Si os da tiempo, empezad el extra: pruebas propias o `plantilla_kpis_pandas.py`.

Mensajes de commit de ejemplo: «Añade lectura del CSV», «Añade KPI y exportación», «Añade README».

Hecho cuando: otra persona podría ejecutar el proyecto siguiendo solo el README. Después, la entrega de 15 minutos: subir a `sprint-04/` y rellenar la autoevaluación y coevaluación.

6 Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **Las pruebas mandan:** si el código de la IA no las pasa, no vale.
- **Declaradlo** en la sección «Uso de IA y límites» del README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender un error	«Me sale este error: *[mensaje completo]*. Explícame qué significa, sin darme todavía la solución»	Que lo entendéis y lo arregláis vosotros
Entender una función	«Explícame línea por línea qué hace esta función: *[código]*»	Ejecutadla con un ejemplo pequeño
Pedir una pista	«Quiero calcular el % de cancelaciones por canal con la biblioteca estándar. ¿Qué estructura me conviene? No me des el código»	Que respeta «solo biblioteca estándar»
Revisar el estilo	«¿Sigue PEP 8 este código? ¿Qué cambiarías?»	Que las pruebas siguen pasando

7 Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
`ModuleNotFoundError: kpis`	Se ejecuta `test_kpis.py` desde otra carpeta	Abrir la terminal **dentro** de la carpeta del reto
La media sale rara	Se incluyen reservas canceladas en el ingreso	**Solo las no canceladas**
La ocupación no sale 42.0	Falta el `+ 1` en los días, se cuentan canceladas o la capacidad no es 40	Releer la fórmula y ejecutar con `--capacidad 40`
Las pruebas fallan todas a la vez	Dependen de `leer_reservas`	Empezar por esa función
El programa se rompe con un fichero mal	No se controlan las excepciones	`try` / `except` con tipos concretos y mensajes claros
Tildes rotas	Falta `encoding="utf8"`	Añadirlo al abrir y escribir ficheros
Cambiar las pruebas para que pasen	Prisa	No tocar las pruebas dadas; añadir las propias aparte
Un solo *commit* gigante	«Lo subo al final»	Commits pequeños mientras avanzáis

8 Si os atascáis

1. Leed el mensaje de la prueba que falla: dice qué esperaba y qué recibió. Probad la función en una celda con un ejemplo pequeño.
2. Repasad el cuaderno de la tabla del paso 2 y su código explicado.
3. Preguntad al equipo y, después, al docente. Git: `S04_05` o subid los ficheros desde la web de GitHub.

9 Cómo se evalúa

Rúbrica del curso, de 0 a 4 en cada criterio.

Criterio	Qué se mirará en este reto
Corrección técnica	Las pruebas pasan, los KPI coinciden con los esperados, los errores están controlados
Reproducibilidad	Otra persona clona el repositorio, sigue el README y obtiene el mismo JSON

Criterio	Qué se mirará en este reto
Análisis y comunicación al cliente	El README y el JSON se entienden sin ser programador; cada KPI está definido y se explica por qué la ocupación es solo un proxy
IA responsable	Se dice qué se hizo con IA y qué se revisó; se citan los límites de los datos sintéticos y del proxy

En la review (3–5 minutos): ejecutad el script con el CSV real, enseñad el JSON, provocad un error (fichero inexistente) y mostrad el mensaje, y enseñad las pruebas en verde. Explicad una decisión de diseño, por ejemplo cómo definisteis la ocupación.

10 Palabras que aparecen en este reto

Palabra	Qué significa, en sencillo
KPI	Una cifra que resume cómo va el negocio
Proxy	Una medida aproximada, usada cuando no se puede medir lo exacto
CSV / JSON	Dos formatos de texto para guardar datos: tabla con comas / datos anidados que entienden otros programas
Biblioteca estándar	Lo que ya trae Python (`csv`, `json`): no hay que instalar nada
Prueba (test)	Código que comprueba que otro código funciona. Usa `assert`: «esto tiene que ser verdad»
Excepción	Un error durante la ejecución que se puede **controlar** para que el programa no se rompa
Docstring / PEP 8	El texto explicativo dentro de cada función / la guía de estilo de Python
Commit	Guardar en Git una versión del trabajo con un mensaje que dice qué cambia

11 Definition of Done (lista de comprobación)

Antes de entregar, repasad esta lista (es la única; sirve también de autoevaluación rápida). Si queda una casilla sin marcar, **el reto no está terminado**.

- [ ] `python test_kpis.py` termina con todas las pruebas superadas.
- [ ] `python kpis.py reservas_turisdata.csv salida.json` genera el JSON con los tres KPI y la sección `calidad`.
- [ ] El script informa con un mensaje claro si falta el fichero o una columna, y descarta y cuenta las filas defectuosas.
- [ ] Todas las funciones tienen docstring y siguen PEP 8 (nombres, sangría).
- [ ] El README explica cómo ejecutar, cómo probar y qué significa cada KPI, incluido por qué la ocupación es un proxy.
- [ ] El repositorio tiene al menos 3 *commits* con mensajes claros y no contiene claves ni datos personales.
- [ ] Alguien del grupo ha revisado el código con la rúbrica.
- [ ] Autoevaluación y coevaluación rellenadas (cada persona sube su hoja).